LuxeCart

E-Commerce Store

Project Documentation

Digital Egypt Pioneers Initiative(

Supervisor: ITC

TeamMembers

Shahd Mansour Abdullah Abu Saada
Basant Roshdy Gouda El-Shahat
Rawan Mahmoud Hassan Ahmed
Doaa Akram Shehab

Jun 2026

1. Introduction

1.1 Project Overview

LuxeCart is a modern e-commerce website featuring a luxury minimalist design. Built strictly as a Frontend Layer using React.js and Tailwind CSS, the platform provides a fast, high-performance, and fully responsive user interface across all devices. The website dynamically fetches product data from the DummyJSON API, utilizes browser LocalStorage to persist shopping cart and wishlist selections locally, and completes the user cycle with a fully simulated client-side Demo Checkout page.

1.2 Problem Statement

Traditional e-commerce websites often suffer from slow performance, cluttered user interfaces, and poor responsiveness across mobile devices, which leads to a frustrating shopping experience. Additionally, many modern web prototypes rely on static visual layouts without demonstrating how frontend applications handle dynamic real-time data or persist user states (like shopping carts and wishlists) during active browser sessions.

1.3 Project Objectives

- **Build a Fully Responsive Design:** Deliver a high-performance web interface optimized for all screen sizes (mobile devices, tablets, and desktops) utilizing a premium, minimalist design approach.
- **Integrate Live Dynamic Data:** Connect the website with an external production REST API (DummyJSON) to dynamically fetch and display real-time categories, prices, and product listings.
- **Enable Local Data Persistence:** Implement persistent shopping cart and wishlist states across active browser sessions and page reloads by leveraging browser LocalStorage.
- **Provide a Simulated Checkout Flow:** Deliver an interactive client-side Demo Checkout page that validates user inputs locally and clears browser storage upon order simulation to finalize the end-to-end user cycle.
- **Develop Scalable and Clean Code:** Organize the React application using modular component structures, custom hooks, and dynamic client-side routing to support future software expansion
-

1.4 Scope

The system covers the full customer-facing journey: browsing products, searching and filtering by category, viewing product details, managing a shopping cart, maintaining a favorites list, and signing in/out. Administrative product management is left as a future extension.

2. Stakeholders & Users

Stakeholder	Role	Primary Interest

Guest User	Visits the site without an account	Browse products, explore categories
Registered Customer	Signed-in shopper	Buy products, manage cart & favorites
Administrator	Manages catalog (future)	Oversee platform scale and catalog planning (Future Extension)
Project Team	Developers & designers	Build & maintain the system
Supervisor	Academic mentor	Evaluate quality & learning outcomes

3. System Requirements

3.1 Functional Requirements

1. The system shall display a list of products fetched from the DummyJSON REST API.
2. The system shall allow users to search products by title, description, or brand.
3. The system shall allow users to filter products by category.
4. The system shall display detailed product information including images, price, rating, stock, and description.
5. The system shall allow users to add products to the shopping cart.
6. The system shall allow users to update item quantities or remove items from the cart.
7. The system shall persist the cart in browser localStorage across sessions.
8. The system shall allow users to add or remove products from a favorites (wishlist) list.
9. The system shall allow users to sign in with a name and email (demo authentication).
10. The system shall display dynamic notification badges in the header for the cart and favorites counts.
11. The system shall allow signed-in users to sign out and clear their session.

3.2 Non-Functional Requirements

Category	Requirement
Performance	Initial page load under 2 seconds on broadband; optimized state management via React Context.
Usability	Mobile-first responsive design; minimum 44x44px touch targets.
Reliability	Graceful error handling for failed API requests with retry logic.
Maintainability	TypeScript strict mode; modular components; reusable custom hooks.
Security	No sensitive data stored client-side; HTTPS-only API calls.

Accessibility	Semantic HTML, ARIA labels, keyboard navigation, sufficient color contrast.
Scalability	Component-based architecture allows easy addition of new pages & features.

4. Technology Stack

Layer	Technology	Purpose
Language	TypeScript 5	Type-safe JavaScript superset
Framework	React 18	Component-based UI library
Build Tool	Vite 5	Lightning-fast dev server & bundler
Styling	Tailwind CSS v3	Utility-first CSS framework
UI Components	shadcn/ui + Radix UI	Accessible, unstyled component primitives
Routing	React Router v6	Client-side routing
Data Fetching	TanStack React Query	Server-state caching & synchronization
Icons	Lucide React	Modern SVG icon set
Notifications	Sonner	Toast notifications
State (local)	React Hooks + localStorage	Cart, favorites, account persistence
External API	DummyJSON REST API	Real product catalog data
Hosting	Lovable Cloud / CDN	Static site deployment
Version Control	Git	Source code management

5. System Architecture

LuxeCart follows a modern client-side architecture pattern. The application is a Single Page Application (SPA) built with React, where all routing and rendering happen in the browser. Data is fetched from an external REST API and managed by React Query, while user-specific state (cart, favorites, account) is persisted to the browser's localStorage.

5.1 Architectural Style

- Component-Based Architecture — UI built from small, reusable components.
- Separation of Concerns — Pages, components, hooks, and API layer are isolated.
- Custom Hooks Pattern — Business logic extracted into testable hooks (useCart, useFavorites, useAccount).
- Single Source of Truth — React Query for server state; localStorage for persistent client state.

5.2 Folder Structure

Folder	Contents
src/pages/	Top-level route components (Home, Shop, Product, Cart, Favorites, Account)
src/components/	Shared UI (Header, Footer) and shadcn/ui primitives
src/hooks/	Custom hooks (useCart, useFavorites, useAccount, use-toast)
src/lib/	API client and utility functions
src/assets/	Static images and AI-generated product visuals
src/index.css	Design tokens (HSL color system, gradients, shadows)

6. UML & System Diagrams

6.1 Use Case Diagram

The use case diagram below shows the main interactions between actors (Guest, Customer, Admin) and the system's primary functions.

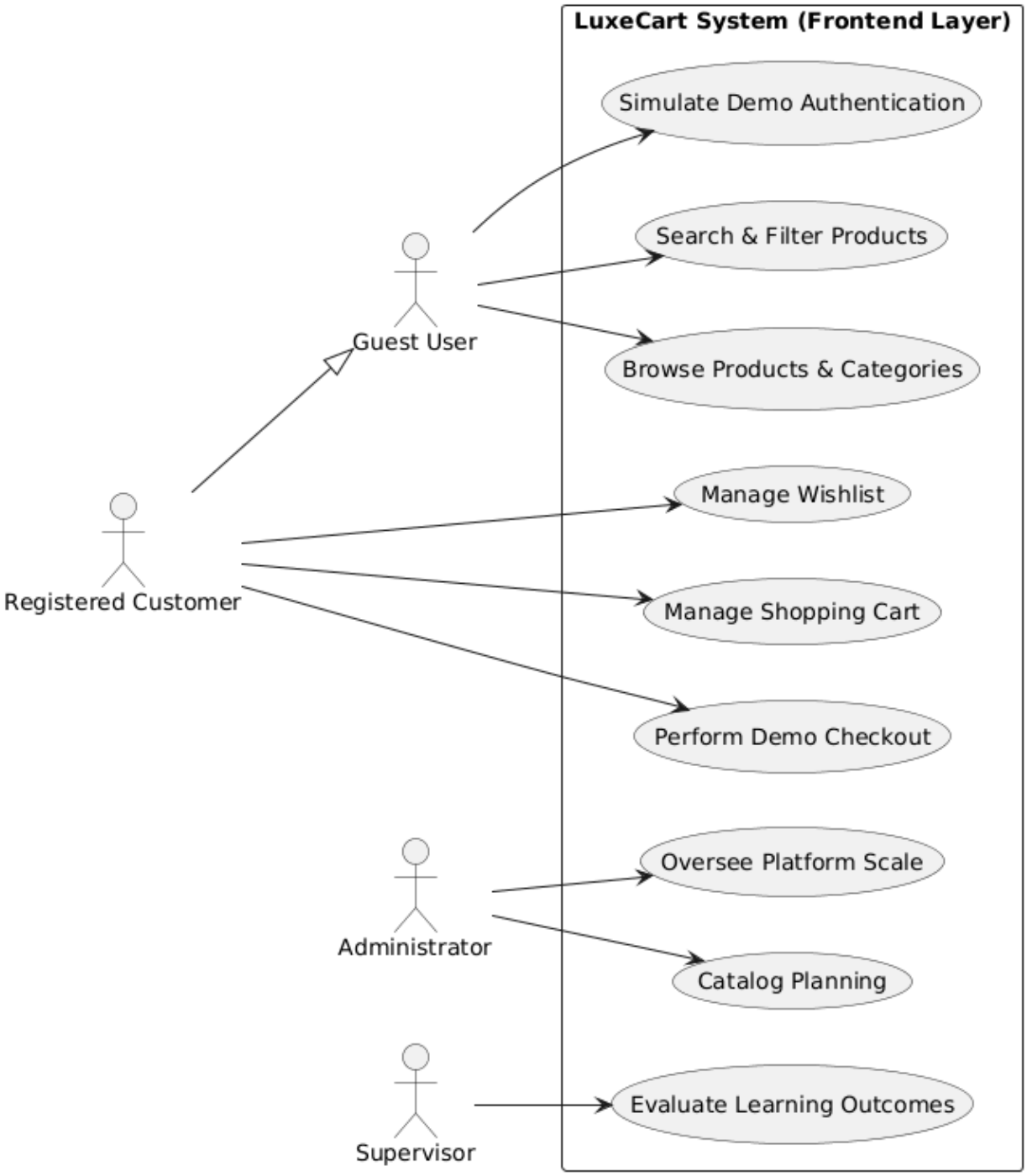

Figure 6.1 — Use Case Diagram

6.2 Class Diagram

The class diagram represents the core domain entities, their attributes, methods, and relationships.

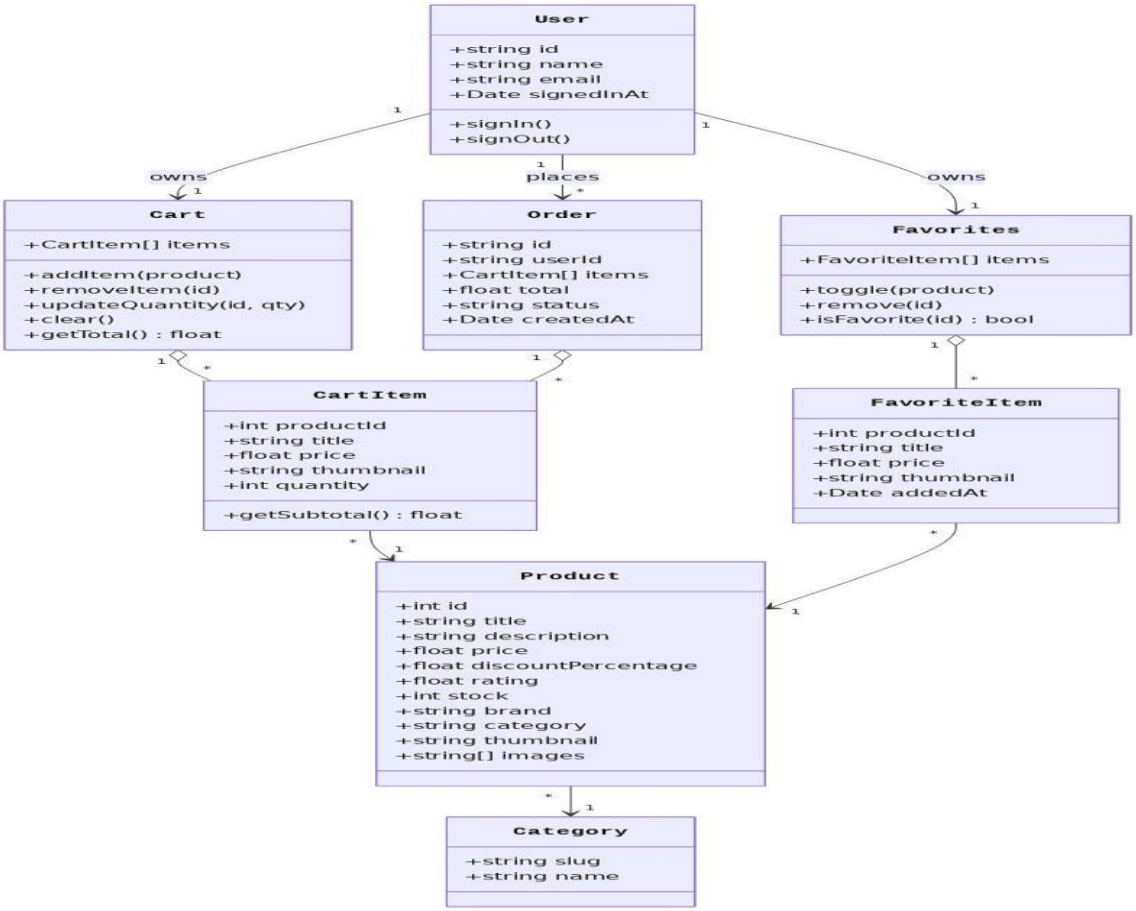

Figure 6.2 — Class Diagram

6.3 Sequence Diagrams

6.3.1 Browse Products Scenario

Shows the flow when a user opens the Shop page: how React Query checks its cache, calls the DummyJSON API on a miss, and renders the result.

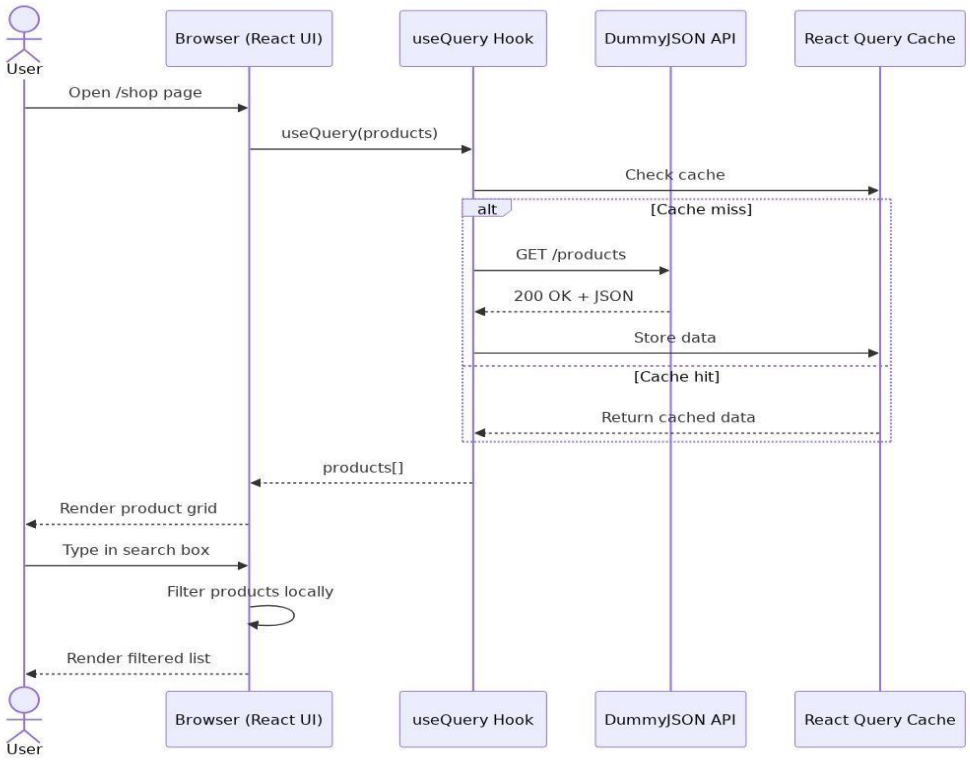

Figure 6.3 — Sequence: Browse Products

6.3.2 Add to Cart Scenario

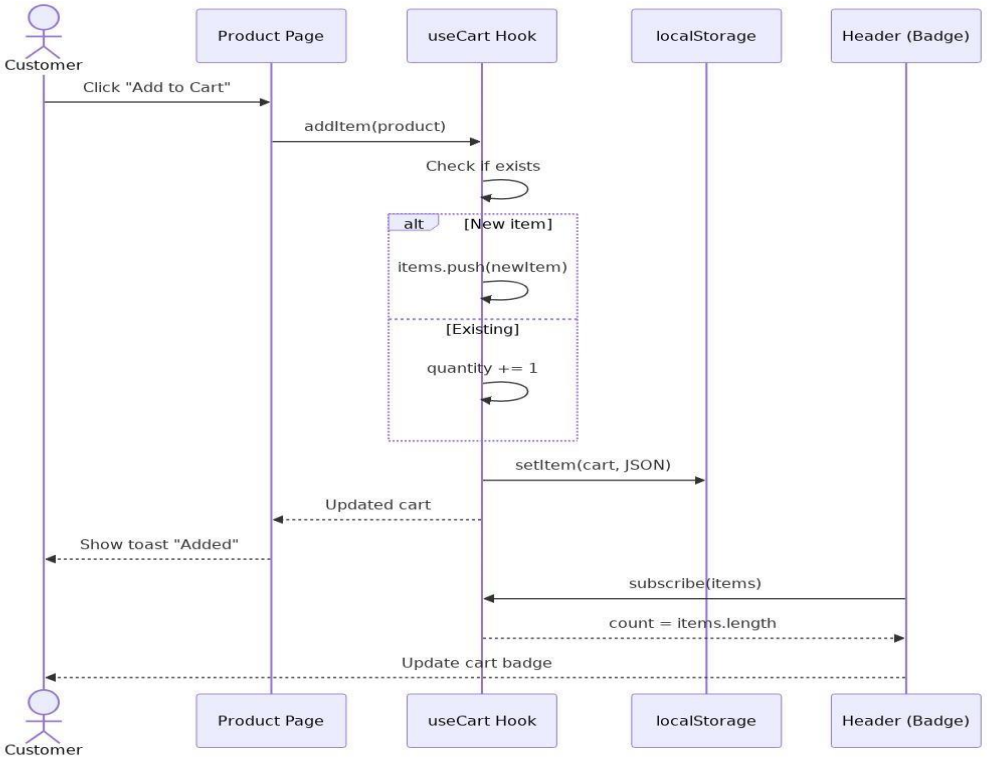

Figure 6.4 — Sequence: Add to Cart

6.3.3 Toggle Favorite Scenario

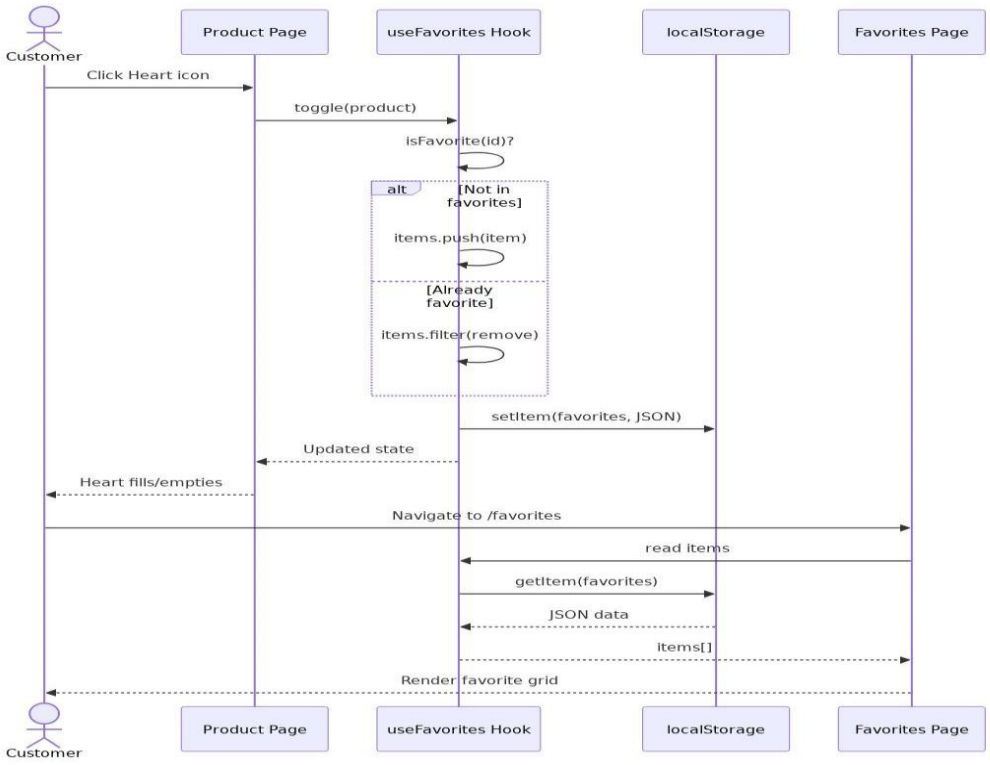

Figure 6.5 — Sequence: Toggle Favorite

6.4 Activity Diagram

End-to-end flow of a customer's shopping journey, from landing on the site to confirming an order.

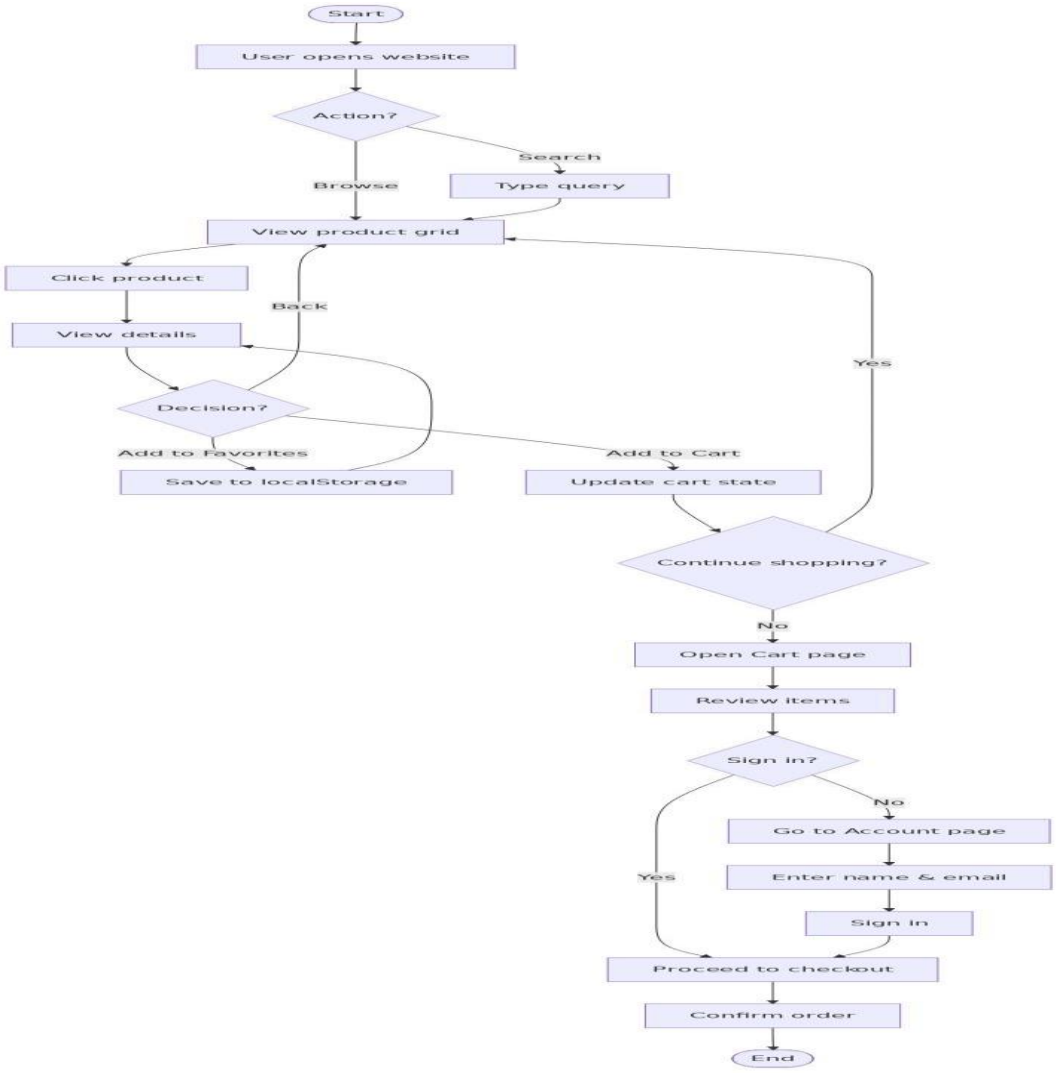

Figure 6.6 — Activity Diagram

6.5 Component Diagram

Logical grouping of frontend modules and their dependencies on external services.

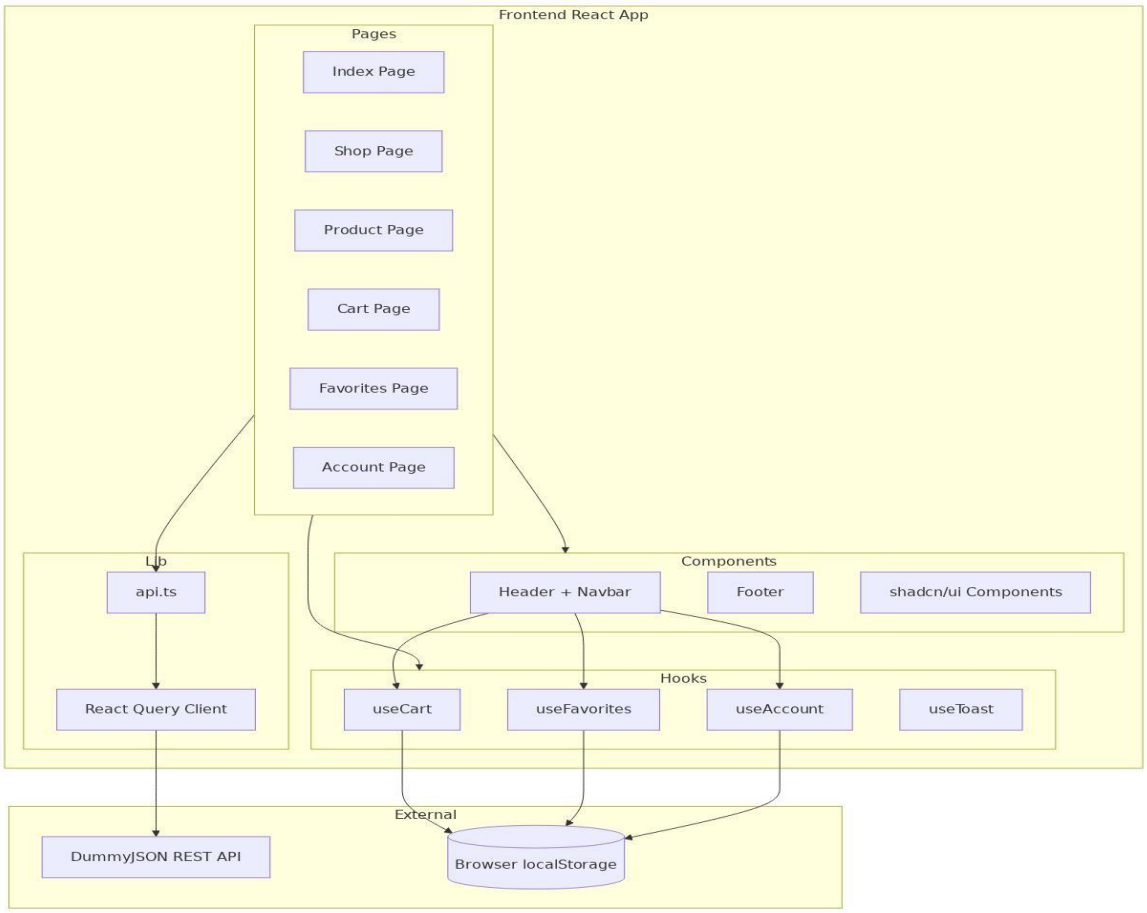

Figure 6.7 — Component Diagram

6.6 Deployment Diagram

Physical view of how the application is deployed and how the client interacts with hosting and external APIs.

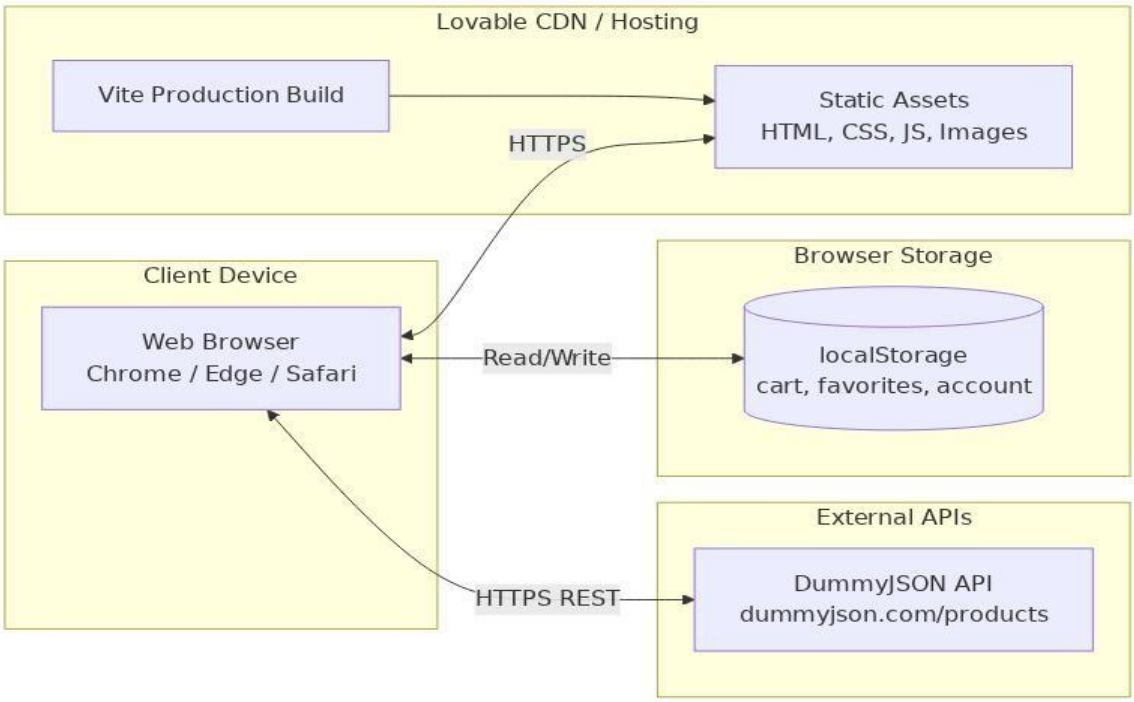

Figure 6.8 — Deployment Diagram

7. Data Design

Although the current implementation uses an external API and localStorage rather than a relational database, a logical ERD is provided to show how the data would be modeled if the system were extended with a backend database (e.g., PostgreSQL via Lovable Cloud). **7.1 Entity Relationship Diagram (ERD)**

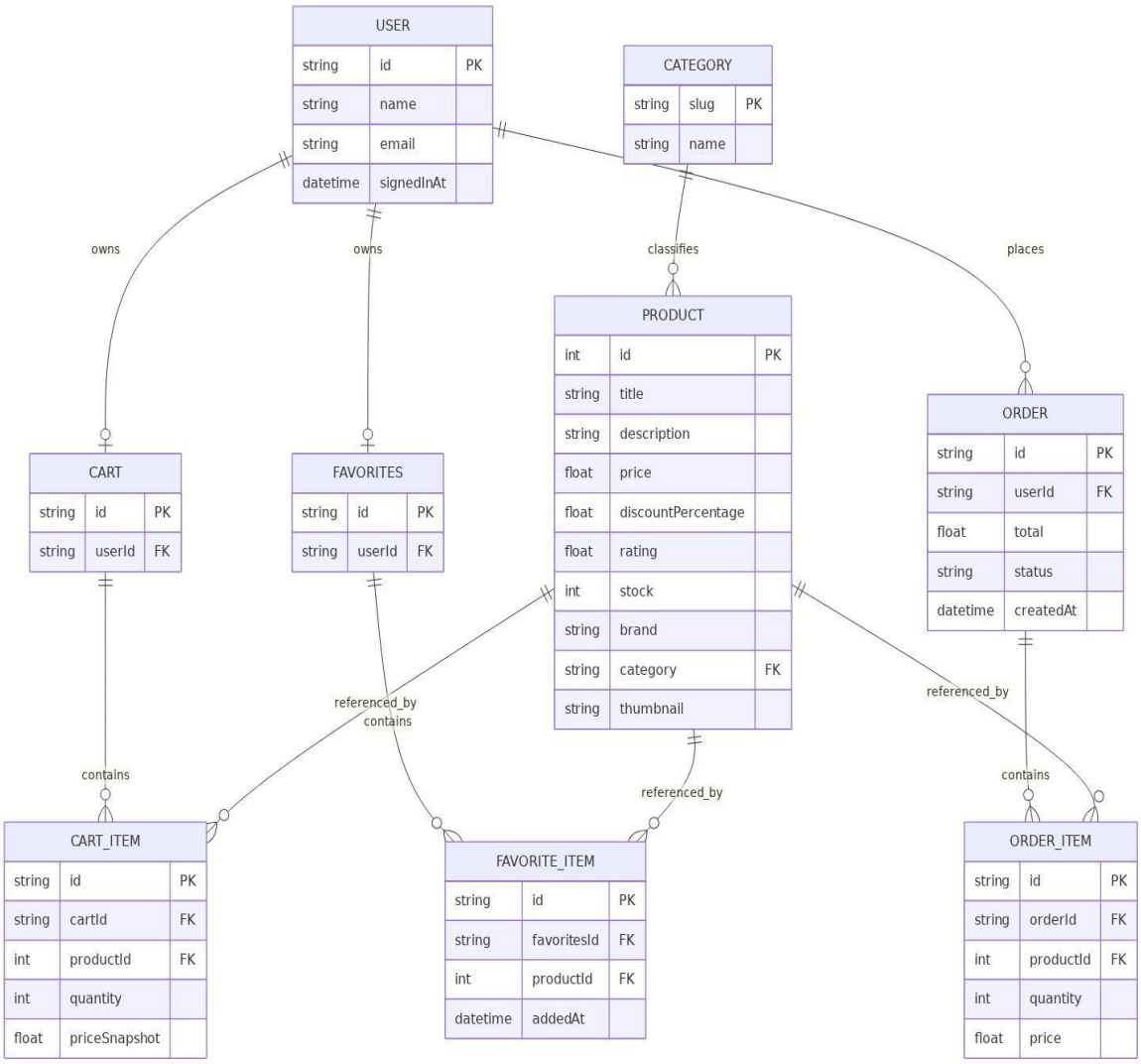

Figure 7.1 — Entity Relationship Diagram

7.2 Data Flow Diagram (DFD — Level 1)

The DFD below illustrates how data flows between the user, system processes, and data stores.

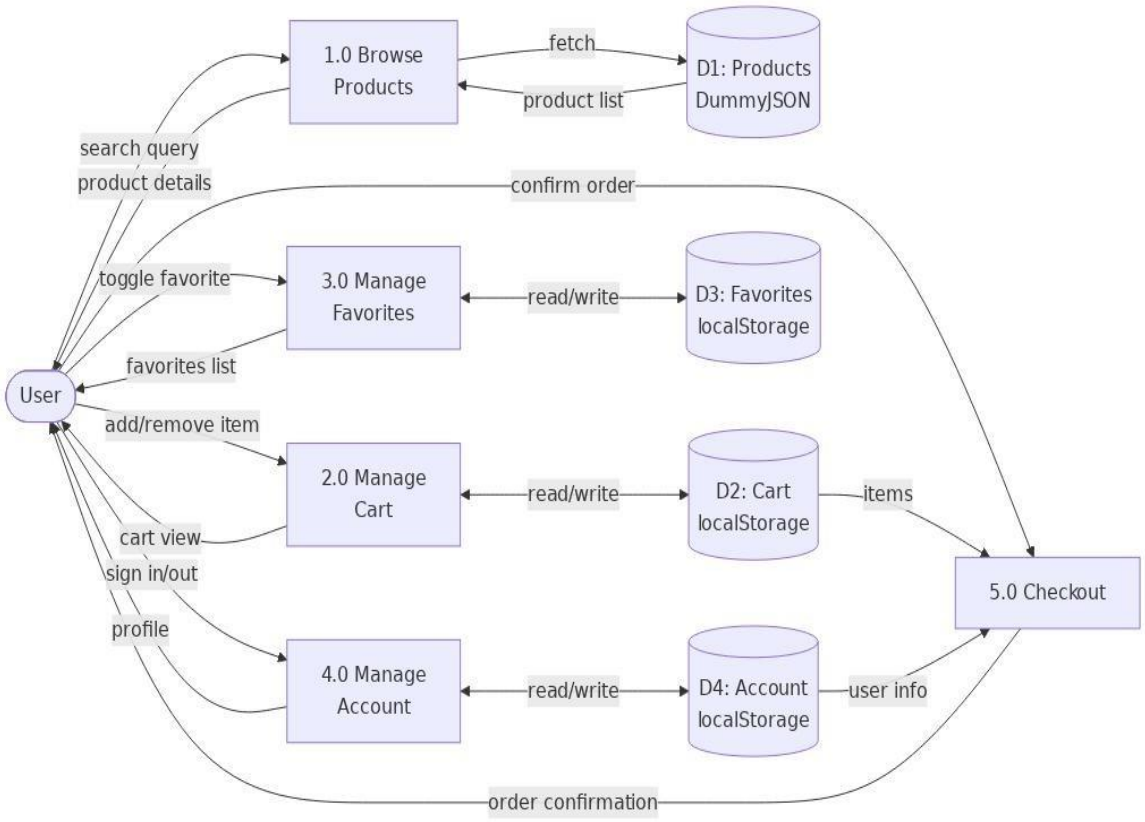

Figure 7.2 — Data Flow Diagram (Level 1)

8. Pages & Features

Page	Route	Description
Home	/	Hero section, featured products, brand storytelling
Shop	/shop	Full product catalog with search and category filter
Product	/product/:id	Product details, images, price, add to cart, favorite toggle
Cart	/cart	Cart items, quantity controls, total, checkout button
Favorites	/favorites	Wishlist grid, move-to-cart, clear all

Account	/account	Sign in form / profile dashboard with cart & favorites stats
404	*	Friendly not-found page

8.1 Key Features

- Real-time product data from DummyJSON REST API.
- Search & filter — instant client-side filtering by title, brand, category.
- Persistent cart — survives page refresh and browser restart via localStorage.
- Favorites system — toggle wishlist items from anywhere in the app.
- Demo authentication — local sign-in with personalized header avatar.
- Dynamic header badges — live counts for cart & favorites.
- Toast notifications — instant feedback for every action.
- Responsive design — works seamlessly on mobile, tablet, and desktop.
- Luxury design system — HSL-based design tokens, custom gradients, elegant typography.

9. UI / UX Design

9.1 Design Philosophy

LuxeCart embraces a minimalist luxury aesthetic. The interface uses generous whitespace, subtle shadows, and a refined neutral palette accented with gold tones. Typography combines a serif display font for headlines with a clean sans-serif for body text, evoking the feel of a premium boutique.

9.2 Color Tokens

Token	Purpose
--background	Page background (warm off-white)
--foreground	Primary text color
--primary	Brand color (deep navy)
--accent	Highlight color (luxury gold)
--muted	Secondary text & borders
--gradient-primary	Hero gradient overlays
--shadow-elegant	Soft elevation for cards

9.3 Responsive Breakpoints

- Mobile: < 640px — single column, stacked navigation.
- Tablet: 640–1024px — two-column grids.
- Desktop: > 1024px — multi-column layouts, side-by-side product views.

10. API Integration

LuxeCart consumes the public DummyJSON REST API to populate its catalog with real product data.

Endpoint	Method	Purpose
/products	GET	Fetch all products
/products/:id	GET	Fetch a single product by ID
/products/search?q=	GET	Search products by query
/products/categories	GET	List all categories
/products/category/:slug	GET	Fetch products in a category

All requests are wrapped in TanStack React Query, which provides automatic caching, background refetching, retry logic, and loading/error states out of the box.

11. Testing & Quality Assurance

- Manual testing across Chrome, Edge, and Safari on desktop and mobile.
- Verified responsive behavior at 320px, 768px, 1024px, and 1440px viewports.
- Tested cart & favorites persistence by refreshing and reopening the browser.
- Validated API error handling by simulating offline mode.
- Accessibility check: keyboard navigation works on all interactive elements.
- TypeScript strict mode catches type errors at compile time.

12. Future Enhancements

- Real authentication (email/password, Google, Apple).
- Persistent backend database (PostgreSQL) replacing localStorage.
- Stripe / Paddle payment integration for real checkout.
- Order history and order tracking pages.
- Admin dashboard for product management.
- Product reviews and star ratings submitted by users.
- Multi-language support (Arabic / English) with RTL layout.
- Email notifications for order confirmation.

13. Conclusion

LuxeCart successfully demonstrates the design and implementation of a modern, professional ecommerce front-end using industry-standard tools. The project applies real-world software engineering practices including modular architecture, type safety, custom hooks, server-state caching, and a polished design system.

The team gained hands-on experience with React, TypeScript, Tailwind CSS, REST API integration, and UI/UX design. The codebase is clean, scalable, and ready to be extended with backend services, payments, and an admin panel as future milestones.

14. References

- React Documentation — https://react.dev
- TypeScript Handbook — https://www.typescriptlang.org/docs/
- Vite — https://vitejs.dev
- Tailwind CSS — https://tailwindcss.com

- shadcn/ui — https://ui.shadcn.com
- TanStack React Query — https://tanstack.com/query
- React Router — https://reactrouter.com
- DummyJSON API — https://dummyjson.com
- Lovable Platform — https://lovable.dev